



北京邮电大学

Beijing University of Posts and Telecommunications

移动终端安全

课程报告

标 题 第二次作业：Android 源代码研究分析与搭建一个
android 操作系统编译环境

姓 名 李昊伦

学 号

班 级

教 师

移动终端安全第二次作业

目 录

一、Android 源代码研究分析	3
1.1 目标.....	3
1.2 核心服务原理分析.....	4
1.2.1 Binder IPC 在 Android 中的作用	4
1.2.2 Binder 与其他 Android 核心机制的关系.....	4
1.2.3 Binder IPC 的总体工作流程.....	5
1.3 源代码分析.....	7
1.3.1 ProcessState：进程级 Binder 初始化.....	7
1.3.2 IPCThreadState：线程级 Binder 事务处理.....	8
1.3.3 BpBinder 与 BBinder：代理对象和本地对象.....	9
1.3.4 Parcel：跨进程数据封装	10
1.3.5 ServiceManager：服务注册和查询.....	11
1.3.6 Binder 驱动：内核态事务转发	12
1.3.7 一次 Binder 调用的完整流程图	13
1.3.8 Binder 的安全机制分析	13
1.4 分析结论.....	14
二、思考题.....	15
2.1 目标.....	15
2.2 相关实验过程.....	15
2.2.1 准备安装环境.....	15
2.2.2 Android 源码下载.....	17
2.2.3 进一步实验，插桩.....	17
2.2.4 配置并编译.....	20
2.2.5 制作最小 initramfs	21
2.2.6 使用 QEMU 启动.....	23
2.2.7 验证插桩日志.....	24
2.3 思考题总结.....	25

一、Android 源代码研究分析

1.1 目标

本报告选择 Android 的 Binder IPC 机制作为核心服务进行源码阅读与分析。Binder IPC 是 Android 系统中最重要进程间通信机制之一。Android 的应用进程、SystemServer、各类系统服务、Native 服务以及 ServiceManager 都大量依赖 Binder 完成跨进程方法调用。与传统 Linux IPC 机制相比，Binder 不只是提供数据传输能力，还将对象引用、服务发现、线程池调度、调用者身份传递等机制整合在一起，因此它是理解 Android 系统架构和安全边界的重要入口。

本报告在阅读过程中参考了课程要求中给出的 AOSP mirror 仓库目录和 AndroidXRef 代码检索网站，并结合 AOSP 官方源码站 android.googlesource.com 对 Binder 相关源码进行了进一步定位与阅读。由于 GitHub 镜像仓库中的部分目录路径可能存在分支变化或页面不可用的问题，因此本文具体源码分析主要以 AOSP 官方源码路径为准。

本报告参考的 Android 源码位置如下：

AOSP mirror: <https://github.com/aosp-mirror>

AndroidXRef: <http://androidxref.com/>

Binder 用户态库：

<https://android.googlesource.com/platform/frameworks/native/+/master/libs/binder/>

ServiceManager：

<https://android.googlesource.com/platform/frameworks/native/+/master/cmds/service-manager/>

Binder 内核驱动：

<https://android.googlesource.com/kernel/common/+/refs/heads/android-mainline/drivers/android/binder.c>

本报告主要阅读的源码文件包括：

[frameworks/native/libs/binder/ProcessState.cpp](#)、

[frameworks/native/libs/binder/IPCThreadState.cpp](#)、

[frameworks/native/libs/binder/BpBinder.cpp](#)、

frameworks/native/libs/binder/Binder.cpp、
frameworks/native/libs/binder/Parcel.cpp、
frameworks/native/cmds/servicemanager/main.cpp、
frameworks/native/cmds/servicemanager/ServiceManager.cpp，以及内核源码
kernel/common/drivers/android/binder.c

1.2 核心服务原理分析

1.2.1 Binder IPC 在 Android 中的作用

Android 是一个多进程系统。普通 App 运行在独立的应用进程中，系统核心服务大多运行在 `system_server` 进程中，部分底层服务运行在 `native daemon` 中，例如 `SurfaceFlinger`、`AudioFlinger`、`MediaServer` 相关服务等。不同进程之间不能直接访问彼此的内存空间，因此需要 IPC 机制完成通信。

Binder IPC 解决的核心问题包括：

1. 服务发现：客户端如何找到系统服务。
2. 跨进程调用：客户端如何像调用本地对象一样调用远程服务。
3. 数据封装：调用参数和返回值如何在进程间传递。
4. 线程调度：服务端如何接收并处理客户端请求。
5. 身份传递：服务端如何知道调用者的 UID、PID，从而进行权限判断。
6. 对象引用管理：跨进程传递的不只是字节数据，还包括 Binder 对象引用。

从使用体验看，用户在手机上打开应用、点击按钮、播放音频、切换界面、请求定位、访问剪贴板等操作，背后都可能触发 Binder 调用。例如应用获取剪贴板内容时，需要通过 Binder 调用系统的 `ClipboardService`；应用播放音频时，需要通过 `AudioFlinger` 相关服务完成音频混音和输出；应用界面刷新时，也会和 `WindowManager`、`SurfaceFlinger` 等服务发生间接或直接联系。

1.2.2 Binder 与其他 Android 核心机制的关系

Binder 不是孤立存在的，它和 Android 的多个核心机制都有关系：

1.与 `Zygote` 的关系：`Zygote` 负责孵化应用进程。应用进程启动后，会通过 Binder 和 `ActivityManagerService`、`PackageManagerService`、`WindowManagerService` 等系统服务通信。因此 `Zygote` 创建的是进程基础，

Binder 支撑的是进程间协作。

2.与 SystemServer 的关系。SystemServer 启动大量 Java 层系统服务，并把它们注册到 ServiceManager。应用进程需要访问这些服务时，先通过 ServiceManager 查询服务，再通过 Binder 代理对象发起调用。

3.与 ServiceManager 的关系。ServiceManager 是 Binder 服务注册中心。服务端调用 addService 注册服务，客户端调用 getService 或 checkService 获取服务引用。ServiceManager 自身也是一个 Binder 服务，通常使用固定 handle 0 作为上下文管理者。

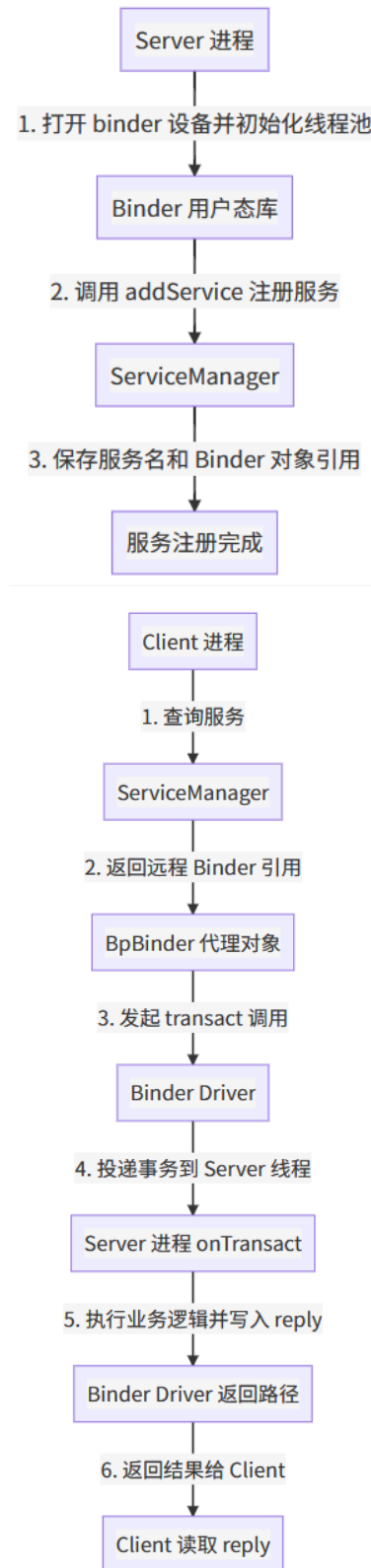
4.与 AudioFlinger、SurfaceFlinger 的关系。AudioFlinger 和 SurfaceFlinger 都属于 Android 的 native 系统服务。应用播放声音或刷新图形界面时，相关请求会通过 Binder 进入对应服务。Binder 提供通道，AudioFlinger 和 SurfaceFlinger 提供具体业务能力。

5.与 LMK、匿名共享内存的关系。LMK 负责低内存场景下的进程回收，匿名共享内存 ashmem 或 memfd 常用于大块数据共享。Binder 适合传递控制命令和少量结构化数据，而大块数据通常通过共享内存传输，再用 Binder 传递文件描述符或引用。这体现了 Android 系统中“控制流走 Binder，数据流可走共享内存”的设计思路。

1.2.3 Binder IPC 的总体工作流程

Binder IPC 通常涉及四类角色：Client 发起调用的客户端进程、Server 提供服务的服务端进程、ServiceManager 负责服务注册和查询、Binder Driver 内核中的 Binder 驱动，负责跨进程数据转发、线程唤醒和引用管理。

整体流程如下：



可以看出，客户端并不直接知道服务端进程的内存地址，也不需要自己处理 socket 连接。Binder 用户态库和 Binder 驱动共同把一次跨进程调用包装成类似“本地对象方法调用”的形式

1.3 源代码分析

1.3.1 ProcessState：进程级 Binder 初始化

源码位置：frameworks/native/libs/binder/ProcessState.cpp。ProcessState 代表一个进程中的 Binder 状态。一个进程通常只需要一个 ProcessState 实例。它负责打开 Binder 驱动、建立 mmap 映射、维护 handle 到代理对象的缓存，并启动 Binder 线程池。

核心逻辑可概括如下：

```
// ProcessState 是每个进程访问 Binder 驱动的入口。
// defaultManager()、IPCThreadState 等路径最终都会依赖它。
sp<ProcessState> ProcessState::self()
{
    // 通过单例方式保证一个进程内共享同一个 ProcessState。
    // 这样可以统一管理 binder fd、mmap 区域和 handle 缓存。
    return init(kDefaultDriver, false);
}

ProcessState::ProcessState(const char* driver)
{
    // 1. 打开 Binder 设备，例如 /dev/binder。
    //     这一步让用户态进程获得和 Binder 驱动通信的 fd。
    mDriverFD = open_driver(driver);

    // 2. 通过 mmap 建立用户态和内核 Binder 驱动共享的映射区域。
    //     Binder 传输数据时，可以减少多次拷贝，提高 IPC 效率。
    mVMStart = mmap(..., mDriverFD, 0);

    // 3. 初始化 handle 缓存。
    //     客户端拿到的远程服务引用会被包装成 BpBinder。
}
```

这部分代码体现出 Binder 的第一个关键设计：Binder 不是单纯的用户态库，而是“用户态库 + 内核驱动”的协作机制。应用进程必须打开 Binder 设备，后续的事务发送、线程等待和结果返回都通过这个 fd 与驱动交互。

1.3.2 IPCThreadState: 线程级 Binder 事务处理

源码位置: frameworks/native/libs/binder/IPCThreadState.cpp。IPCThreadState 代表当前线程的 Binder 通信状态。它内部维护两个重要缓冲区: mOut 和 mIn。mOut 用于写入要发送给 Binder 驱动的命令, mIn 用于接收驱动返回的命令或数据。

一次典型客户端调用会进入 transact():

```
status_t IPCThreadState::transact(int32_t handle,
                                uint32_t code,
                                const Parcel& data,
                                Parcel* reply,
                                uint32_t flags)
{
    // 1. 将目标 handle、调用编号 code、参数 data
    //    打包成 Binder 驱动可以理解的 transaction_data。
    writeTransactionData(BC_TRANSACTION, flags, handle, code, data, nullptr);
    // 2. 如果不是 one-way 异步调用, 就等待服务端返回结果。
    //    talkWithDriver() 会通过 ioctl 进入内核 Binder 驱动。
    if ((flags & TF_ONE_WAY) == 0) {
        waitForResponse(reply);
    } else {
        waitForResponse(nullptr, nullptr);
    }
    return err;
}
```

其中 talkWithDriver() 是用户态进入内核的关键函数:

```
status_t IPCThreadState::talkWithDriver(bool doReceive)
{
    // 1. 准备 binder_write_read 结构。
    //    write 部分对应 mOut, read 部分对应 mIn。
```



```

binder_write_read bwr;

bwr.write_buffer = (uintptr_t)mOut.data();

bwr.write_size = mOut.dataSize();

bwr.read_buffer = (uintptr_t)mIn.data();

bwr.read_size = mIn.dataCapacity();

// 2. 通过 ioctl(BINDER_WRITE_READ) 和 Binder 驱动通信。
//     驱动会处理 BC_TRANSACTION、BC_REPLY 等命令，
//     并向用户态返回 BR_TRANSACTION、BR_REPLY 等结果。
ioctl(mProcess->mDriverFD, BINDER_WRITE_READ, &bwr);

// 3. 更新 mOut 和 mIn 的读写位置，供后续解析。
}

```

这说明 Binder 调用并不是“用户态直接调用服务端函数”，而是把调用请求编码为命令，交给内核驱动调度。内核驱动再把请求投递给目标进程中的 Binder 线程。

1.3.3 BpBinder 与 BBinder：代理对象和本地对象

源码位置：frameworks/native/libs/binder/BpBinder.cpp。Android Binder 采用代理模式。客户端看到的是 BpBinder，服务端真正实现的是 BBinder 或其派生类。

BpBinder::transact() 的职责是把客户端调用继续转交给 IPCThreadState::transact()。

```

status_t BpBinder::transact(uint32_t code,
                             const Parcel& data,
                             Parcel* reply,
                             uint32_t flags)
{
    // BpBinder 不直接处理业务逻辑。
    // 它只保存远程对象 handle，并把请求交给 IPCThreadState。
    return IPCThreadState::self()->transact(mHandle, code, data, reply, flags);
}

```

服务端收到请求后，会进入 `BBinder::transact()`，再分发到 `onTransact()`。

```
status_t BBinder::transact(uint32_t code,
                           const Parcel& data,
                           Parcel* reply,
                           uint32_t flags)
{
    // 服务端本地 Binder 对象根据 code 判断调用哪个接口。
    // Java Binder 或 AIDL 生成的 Stub 本质上也会走类似分发逻辑。
    return onTransact(code, data, reply, flags);
}
```

因此，一次 Binder 调用的对象模型可以总结为：Client 调用接口方法、Proxy/BpBinder、IPCThreadState::transact()、Binder Driver、Server Binder Thread、Stub / BBinder::onTransact()、真正的服务实现函数

1.3.4 Parcel：跨进程数据封装

源码位置：frameworks/native/libs/binder/Parcel.cpp。Binder 需要传输调用参数和返回值。Android 使用 Parcel 作为序列化容器。客户端把基本类型、字符串、文件描述符、Binder 对象引用等写入 Parcel，服务端按相同顺序读取。

```
// 客户端侧：写入调用参数
Parcel data;
data.writeInterfaceToken(interfaceName);
data.writeInt32(userId);
data.writeString16(packageName);
remote->transact(TRANSACTION_XXX, data, &reply);

// 服务端侧：读取调用参数
data.enforceInterface(interfaceName);
int32_t userId = data.readInt32();
String16 packageName = data.readString16();
```

Parcel 的安全意义在于：跨进程调用不能传递裸指针，因为指针只在本进程地址空间有意义。Binder 必须把数据转换为可复制、可校验、可重建的格式。对

于文件描述符和 Binder 对象引用，Binder 驱动会进行特殊处理，确保接收方拿到的是合法引用，而不是任意内存地址。

1.3.5 ServiceManager：服务注册和查询

源码位置：frameworks/native/cmds/servicemanager/main.cpp、frameworks/native/cmds/servicemanager/ServiceManager.cpp。

ServiceManager 是 Binder 服务的注册中心。服务端把自己注册进去，客户端通过服务名查询。它类似 Android native 层的“电话簿”。

核心流程如下：ServiceManager 启动、打开 Binder 驱动 ProcessState::self()、设置自己为 context manager、成为 handle 0 对应的服务管理者、进入 Binder loop、等待 addService/getService/checkService 请求

服务注册可概括为：

```
Status ServiceManager::addService(const std::string& name,
                                   const sp<IBinder>& binder,
                                   bool allowIsolated,
                                   int32_t dumpPriority)
{
    // 1. 检查服务名是否合法。
    // 2. 检查调用者是否有注册该服务的权限。
    // 3. 保存 name -> binder 的映射关系。
    // 4. 如果服务死亡，借助 linkToDeath 清理状态。
    mNameToService[name] = Service { binder, allowIsolated, dumpPriority };
    return Status::ok();
}
```

服务查询可概括为：

```
sp<IBinder> ServiceManager::checkService(const std::string& name)
{
    // 1. 根据 name 查找服务表。
    // 2. 如果存在，返回对应 Binder 引用。
    // 3. 客户端拿到引用后，会在本进程中包装成代理对象。
}
```

```
        return service.binder;
    }
```

从安全角度看，ServiceManager 不只是简单字典。它还需要结合 SELinux 和调用者身份进行访问控制，防止普通应用注册伪造系统服务，或者访问不应访问的底层服务。

1.3.6 Binder 驱动：内核态事务转发

源码位置：kernel/common/drivers/android/binder.c。Binder 驱动是整个机制的核心。用户态库通过 ioctl(BINDER_WRITE_READ)把命令交给驱动。驱动解析命令后，创建事务对象，找到目标进程和目标线程，把请求放入目标线程或进程的工作队列，并唤醒服务端 Binder 线程。

驱动核心处理过程可概括如下：

```
// 用户态通过 ioctl 进入 Binder 驱动。
static long binder_ioctl(struct file *filp,
                        unsigned int cmd,
                        unsigned long arg)
{
    switch (cmd) {
        case BINDER_WRITE_READ:
            // 处理用户态写入的命令，并把内核返回的数据写回用户态。
            binder_ioctl_write_read(filp, cmd, arg, thread);
            break;
    }
}
```

当驱动收到 BC_TRANSACTION 命令时，会进入事务处理逻辑：

```
static void binder_transaction(...)
{
    // 1. 根据 handle 找到目标 Binder node。
    //     handle 是客户端看到的远程对象编号，
    //     驱动内部会映射到服务端真实 Binder 实体。
```

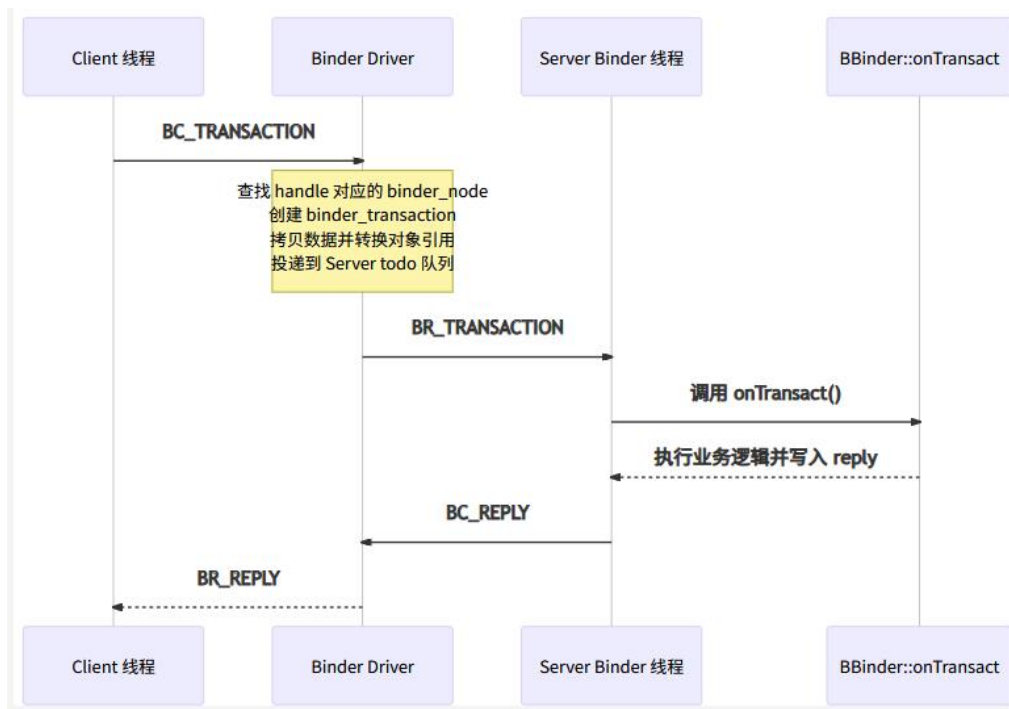
```

// 2. 分配 binder_transaction 结构,
//     保存调用 code、flags、发送方信息、目标信息等。
// 3. 拷贝或翻译 Parcel 中的数据。
//     普通数据需要复制;
//     Binder 对象和文件描述符需要转换成接收进程可理解的引用。
// 4. 把 transaction 放入目标线程或目标进程的 todo 队列。
// 5. 唤醒目标 Binder 线程。
//     服务端线程之后会从驱动读取 BR_TRANSACTION。
}

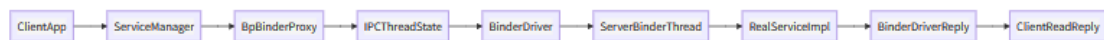
```

服务端线程被唤醒后，会从驱动读到 BR_TRANSACTION，然后用户态 IPCThreadState::executeCommand() 解析命令并调用本地 Binder 对象。处理完成后，服务端通过 BC_REPLY 把结果交回驱动，驱动再把 BR_REPLY 返回给客户端。

完整时序如下：



1.3.7 一次 Binder 调用的完整流程图



1.3.8 Binder 的安全机制分析

- 1.调用者身份传递 **Binder** 驱动知道发起事务的进程和线程，因此服务端可以通过 **Binder API** 获取调用者 **UID/PID**。系统服务常用调用者 **UID** 判断权限，例如判断某个 **App** 是否有访问定位、相机、剪贴板或账户数据的权限。
- 2.服务访问控 **ServiceManager** 对服务注册和查询进行权限检查，结合 **SELinux** 限制不同域之间的访问。这样可以防止普通应用伪造系统服务，也可以限制敏感 **native** 服务的暴露范围。
- 3.对象引用由内核管理跨进程传递 **Binder** 对象时，客户端拿到的是 **handle**，而不是服务端内存地址。**handle** 到真实对象的映射由 **Binder** 驱动维护，降低了直接伪造对象地址的风险。
- 4.支持死亡通知 **Binder** 支持 **linkToDeath**。当服务端进程死亡时，客户端可以收到死亡通知并清理状态。系统服务也可以用它监控客户端生命周期。
- 5.降低 **IPC** 使用复杂度。**Android** 上大量系统服务需要频繁跨进程通信。如果每个服务都自己实现 **socket** 协议、身份校验和线程池，安全风险会更高。**Binder** 提供统一基础设施，有利于集中维护权限和对象引用规则。

1.4 分析结论

通过阅读 **Binder IPC** 相关源码，可以看出 **Android** 并不是简单地在 **Linux** 上堆叠应用框架，而是在 **Linux** 内核能力基础上构建了一套面向移动终端的系统服务架构。**Binder** 是这套架构的核心连接机制。

第一，**Binder** 体现 **Android** 的分层设计。应用层通过 **Java Framework** 调用系统能力，**Framework** 层通过 **Binder** 进入 **SystemService** 或 **native** 服务，底层再通过 **HAL**、内核驱动访问硬件。用户平时点击屏幕、播放音乐、切换应用等操作，背后都有大量 **Binder** 调用在不同进程之间流转。

第二，**Binder** 体现了 **Android** 的安全设计。**Android** 用 **UID** 区分应用沙箱，用权限系统控制敏感能力，用 **SELinux** 控制进程域访问，而 **Binder** 是这些安全信息传递的重要通道。系统服务之所以能判断“是谁在调用我”，很大程度上依赖 **Binder** 驱动提供的调用者身份。

第三，**Binder** 体现了移动系统对性能的考虑。移动终端资源有限，跨进程通信既要安全，又不能过度消耗 **CPU** 和内存。**Binder** 通过内核驱动统一调度事务，通过 **mmap** 缓冲区减少数据复制，并支持对象引用传递和线程池处理，从而适合

Android 中高频的系统服务调用场景。

第四，Binder 也说明 Android 内核和普通 Linux 内核的关注点不同。普通 Linux IPC 更偏向通用机制，而 Android Binder 面向系统服务模型做了深度定制。它把服务发现、对象代理、权限身份、线程唤醒和生命周期管理结合起来，形成了 Android 独特的系统通信基础。

综合来看，Binder IPC 是理解 Android 内核和系统服务关系的关键。它连接了应用、Framework、SystemService、native 服务和内核驱动。阅读 Binder 源码后，我对 Android 的认识从“手机上的应用运行平台”进一步变成了“以进程隔离为基础、以 Binder 为通信骨架、以系统服务为能力出口的移动操作系统”。在移动终端安全课程中，分析 Binder 有助于理解权限检查、服务隔离、系统调用链路和进程间攻击面的形成原因。

二、思考题

2.1 目标

搭建一个 android 操作系统编译环境，下载任意一个版本的 android 原生系统源代码，实现某一个核心功能的插桩，使其输出日志信息（例如修改网络读写、文件读写、访问 GPS 等模块的核心代码，当访问网络、文件或者位置时输出日志记录），并进行编译，使其能够在 qemu 虚拟机或者真机中启动及运行，并将修改后的日志输出到命令行或 ADB 中。

2.2 相关实验过程

2.2.1 准备安装环境

1. 安装编译依赖

```
sudo apt-get update
sudo apt-get install -y \
    build-essential bc bison flex libssl-dev libelf-dev \
    libncurses-dev dwarves pahole cpio gzip rsync file git curl \
    qemu-system-x86 busybox-static
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ sudo apt-get update
[sudo] password for ubuntu:
adSorry, try again.
[sudo] password for ubuntu:
Sorry, try again.
[sudo] password for ubuntu:
Hit:1 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
Reading package lists... Done
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ sudo apt-get install -y \
> build-essential bc bison flex libssl-dev libelf-dev \
> libncurses-dev dwarves pahole cpio gzip rsync file git curl \
> qemu-system-x86 busybox-static
Reading package lists... Done
Building dependency tree
Reading state information... Done
5 packages to be installed, 0 to be removed.
```

```
Setting up busybox-static (1.1.30-1-4ubuntu0.3) ...
Setting up dwarves (1.21-0ubuntu1~20.04.1) ...
Setting up rsync (3.1.3-8ubuntu0.9) ...
Processing triggers for systemd (245.4-4ubuntu3.17) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for initramfs-tools (0.136ubuntu6.7) ...
update-initramfs: Generating /boot/initrd.img-5.15.0-50-generic
```

2.验证工具链

```
# 验证 gcc 版本
gcc --version
# 验证 make 版本
make --version
# 验证 git 版本
git --version
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ gcc --version
gcc (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0
Copyright (C) 2019 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

ubuntu@ubuntu-virtual-machine:~/Desktop$ # 验证make版本
ubuntu@ubuntu-virtual-machine:~/Desktop$ make --version
GNU Make 4.2.1
Built for x86_64-pc-linux-gnu
Copyright (C) 1988-2016 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

ubuntu@ubuntu-virtual-machine:~/Desktop$ # 验证git版本
ubuntu@ubuntu-virtual-machine:~/Desktop$ git --version
git version 2.25.1
```

3.创建工作目录

```
cd ~/Desktop
```



```
mkdir -p ~/android-kernel-lab
cd ~/android-kernel-lab
```

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ mkdir -p ~/android-kernel-lab
ubuntu@ubuntu-virtual-machine:~/Desktop$ cd ~/android-kernel-lab
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$
```

2.2.2 Android 源码下载

1. 下载源码

```
cd ~/android-kernel-lab
git clone --depth 1 --branch android13-5.15 \
    https://android.googlesource.com/kernel/common android-kernel
cd android-kernel
```

2. 验证源码

```
pwd
ls
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ pwd
/home/ubuntu/android-kernel-lab
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ls
android                  common-refs_heads_android13-5.15.tar.gz
arch                    COPYING
block                   CREDITS
BUILD.bazel             crypto
build.config.aarch64    Documentation
build.config.allmodconfig drivers
build.config.allmodconfig.aarch64 fs
build.config.allmodconfig.arm include
build.config.allmodconfig.x86_64 init
build.config.amlogic    io_uring
build.config.arm         ipc
build.config.common      Kbuild
build.config.constants   Kconfig
build.config.db845c       Kconfig.ext
build.config.gce.x86_64   kernel
build.config.gki          lib
build.config.gki.aarch64  LICENSES
build.config.gki.aarch64.fips140 MAINTAINERS
build.config.gki-debug.aarch64 Makefile
build.config.gki-debug.x86_64 mm
build.config.gki_kasan    net
build.config.gki_kasan.aarch64 OWNERS
build.config.gki_kasan.x86_64 README
build.config.gki_kprobes  README.md
build.config.gki_kprobes.aarch64 samples
build.config.gki_kprobes.x86_64 scripts
build.config.gki.x86_64   security
build.config.khwasan      sound
build.config.rockpi4      tools
build.config.x86_64       usr
certs                     virt
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$
```

2.2.3 进一步实验，插桩

1. 修改源码，先备份文件

```
cd ~/android-kernel-lab/android-kernel
cp net/ipv4/tcp.c net/ipv4/tcp.c.bak
```

2.插入日志。本实验选择对 Linux 网络读写模块进行插桩，在 net/ipv4/tcp.c 的 tcp_sendmsg_locked() 与 tcp_recvmsg() 函数中加入 pr_info() 日志。当系统发生 TCP 数据发送或接收时，内核会输出进程编号、进程名以及发送/接收数据长度等信息，用于记录网络访问行为。

```
python3 - <<'PY'
from pathlib import Path

p = Path("net/ipv4/tcp.c")
s = p.read_text()

old1 = """\tbool zc = false;
\tlong timeo;
"""\n
new1 = """\tbool zc = false;
\tlong timeo;

\tpr_info("[NET-TCP-SEND] pid=%d comm=%s size=%zu\\n",
\t\tcurrent->pid, current->comm, size);
"""\n
if old1 not in s:
    raise SystemExit("send insert point not found")
s = s.replace(old1, new1, 1)

old2 = """\tstruct sk_buff *skb, *last;
\tu32 urg_hole = 0;

\tterr = -ENOTCONN;
"""\n
new2 = """\tstruct sk_buff *skb, *last;
\tu32 urg_hole = 0;

\tpr_info("[NET-TCP-RECV] pid=%d comm=%s len=%zu\\n",
\t\tcurrent->pid, current->comm, len);
\tterr = -ENOTCONN;
"""\n
```

if old2 not in s:

raise SystemExit("recv insert point not found")

s = s.replace(old2, new2, 1)

p.write_text(s)

PY

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ python3 - <<'PY'
> from pathlib import Path
>
> p = Path("net/ipv4/tcp.c")
> s = p.read_text()
>
> old1 = """\tbool zc = false;
> \tlong timeo;
> """
> new1 = """\tbool zc = false;
> \tpr_info("[NET-TCP-SEND] pid=%d comm=%s size=%zu\\n",
> \t\tcurrent->pid, current->comm, size);
> \tlong timeo;
> """
> if old1 not in s:
>     raise SystemExit("send insert point not found")
> s = s.replace(old1, new1, 1)
>
> old2 = """\tstruct sk_buff *skb, *last;
> \tu32 urg_hole = 0;
>
> \terr = -ENOTCONN;
> """
> new2 = """\tstruct sk_buff *skb, *last;
> \tu32 urg_hole = 0;
>
> \tpr_info("[NET-TCP-RCV] pid=%d comm=%s len=%zu\\n",
> \t\tcurrent->pid, current->comm, len);
> \terr = -ENOTCONN;
> """
> if old2 not in s:
>     raise SystemExit("recv insert point not found")
```

3.检查修改

grep -n "NET-TCP" net/ipv4/tcp.c

diff -u net/ipv4/tcp.c.bak net/ipv4/tcp.c

```

ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ grep -n "NET-TCP" net/ipv4/tcp.c
1221:  pr_info("[NET-TCP-SEND] pid=%d comm=%s size=%zu\n",
2327:  pr_info("[NET-TCP-RECV] pid=%d comm=%s len=%zu\n",
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ diff -u net/ipv4/tcp.c.bak net/ipv4/tcp.c
--- net/ipv4/tcp.c.bak  2026-04-24 05:29:52.190037979 -0700
+++ net/ipv4/tcp.c      2026-04-25 03:32:57.980624150 -0700
@@ -1218,6 +1218,8 @@
     int mss_now = 0, size_goal, copied_syn = 0;
     int process_backlog = 0;
     bool zc = false;
+    pr_info("[NET-TCP-SEND] pid=%d comm=%s size=%zu\n",
+           current->pid, current->comm, size);
     long timeo;

     flags = msg->msg_flags;
@@ -2322,6 +2324,8 @@
     struct sk_buff *skb, *last;
     u32 urg_hole = 0;

+    pr_info("[NET-TCP-RECV] pid=%d comm=%s len=%zu\n",
+           current->pid, current->comm, len);
     err = -ENOTCONN;
     if (sk->sk_state == TCP_LISTEN)
         goto out;
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$

```

2.2.4 配置并编译

生成基础配置并编译

```

make x86_64_defconfig
make -j$(nproc)

```

```

ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ make x86_64_defconfig
HOSTCC  scripts/basic/fixdep
HOSTCC  scripts/kconfig/conf.o
HOSTCC  scripts/kconfig/confdata.o
HOSTCC  scripts/kconfig/expr.o
LEX      scripts/kconfig/lexer.lex.c
YACC     scripts/kconfig/parser.tab.[ch]
HOSTCC  scripts/kconfig/lexer.lex.o
HOSTCC  scripts/kconfig/menu.o
HOSTCC  scripts/kconfig/parser.tab.o
HOSTCC  scripts/kconfig/preprocess.o
HOSTCC  scripts/kconfig/symbol.o

```

```

SYSHDR arch/x86/include/generated/uapi/asm/unistd_64.h
SYSHDR arch/x86/include/generated/uapi/asm/unistd_x32.h
WRAP arch/x86/include/generated/uapi/asm/bpf_perf_event.h
WRAP arch/x86/include/generated/uapi/asm/errno.h
WRAP arch/x86/include/generated/uapi/asm/fcntl.h
WRAP arch/x86/include/generated/uapi/asm/ioctl.h
WRAP arch/x86/include/generated/uapi/asm/ioctls.h
SYSTBL arch/x86/include/generated/asm/syscalls_32.h
WRAP arch/x86/include/generated/uapi/asm/ipcbuf.h
WRAP arch/x86/include/generated/uapi/asm/param.h
WRAP arch/x86/include/generated/uapi/asm/poll.h
WRAP arch/x86/include/generated/uapi/asm/resource.h
WRAP arch/x86/include/generated/uapi/asm/socket.h
WRAP arch/x86/include/generated/uapi/asm/sockios.h
WRAP arch/x86/include/generated/uapi/asm/termbits.h
WRAP arch/x86/include/generated/uapi/asm/termios.h
WRAP arch/x86/include/generated/uapi/asm/types.h
SYSHDR arch/x86/include/generated/asm/unistd_32_ia32.h
SYSHDR arch/x86/include/generated/asm/unistd_64_x32.h
SYSTBL arch/x86/include/generated/asm/syscalls_64.h
HOSTCC arch/x86/tools/relocs_32.o
UPD include/config/kernel.release
HOSTCC arch/x86/tools/relocs_64.o
WRAP arch/x86/include/generated/asm/early_ioremap.h
WRAP arch/x86/include/generated/asm/export.h
WRAP arch/x86/include/generated/asm/mcs_spinlock.h
WRAP arch/x86/include/generated/asm/irq_regs.h
WRAP arch/x86/include/generated/asm/kmap_size.h
WRAP arch/x86/include/generated/asm/local64.h
WRAP arch/x86/include/generated/asm/mmiowb.h
WRAP arch/x86/include/generated/asm/module.lds.h
WRAP arch/x86/include/generated/asm/rwonce.h
WRAP arch/x86/include/generated/asm/unaligned.h
UPD include/generated/uapi/linux/version.h
UPD include/generated/utsrelease.h
HOSTCC arch/x86/tools/relocs_common.o
HOSTCC scripts/selinux/genheaders/genheaders

```

3. 验证

```
ls -lh arch/x86/boot/bzImage
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ls -lh arch/x86/boot/bzImage
-rw-rw-r-- 1 ubuntu ubuntu 11M Apr 25 05:58 arch/x86/boot/bzImage
```

```
grep -n "NET-TCP-SEND" net/ipv4/tcp.c
```

```
grep -n "NET-TCP-RCV" net/ipv4/tcp.c
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ grep -n "NET-TCP-SEND" net/i
pv4/tcp.c
1223: pr_info("[NET-TCP-SEND] pid=%d comm=%s size=%zu\n",
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ grep -n "NET-TCP-RCV" net/i
pv4/tcp.c
2328: pr_info("[NET-TCP-RCV] pid=%d comm=%s len=%zu\n",
```

2.2.5 制作最小 initramfs

为了让内核在 QEMU 中启动后能进入一个可操作的 shell，本实验自制一个最小根文件系统。

1.创建目录

```
cd ~/android-kernel-lab
mkdir -p initramfs/{bin,sbin,etc,proc,sys,usr/bin,usr/sbin,dev}
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ cd ~/android-kernel-lab
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ mkdir -p initramfs/{bin,sbin,etc,proc,sys,usr/bin,usr/sbin,dev}
```

2.准备 BusyBox

```
cp /usr/bin/busybox initramfs/bin/
chmod +x initramfs/bin/busybox
ln -sf busybox initramfs/bin/sh
ln -sf busybox initramfs/bin/mount
ln -sf busybox initramfs/bin/echo
ln -sf busybox initramfs/bin/cat
ln -sf busybox initramfs/bin/dmesg
ln -sf busybox initramfs/bin/ping
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ which busybox
/usr/bin/busybox
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ cp /usr/bin/busybox initramfs/bin/
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ chmod +x initramfs/bin/busybox
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ln -sf busybox initramfs/bin/sh
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ln -sf busybox initramfs/bin/mount
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ln -sf busybox initramfs/bin/echo
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ln -sf busybox initramfs/bin/cat
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ln -sf busybox initramfs/bin/dmesg
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ln -sf busybox initramfs/bin/ping
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$
```

3.编写/init

```
cat > ~/android-kernel-lab/initramfs/init <<'EOF'
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs devtmpfs /dev
echo
echo "===== "
echo " minimal initramfs booted"
echo " run: dmesg | grep NET-TCP"
echo "===== "
```

```
echo
exec /bin/sh
EOF
chmod +x ~/android-kernel-lab/initramfs/init
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ cat > ~/android-kernel-lab/initramfs/init <<'EOF'
> #!/bin/sh
> mount -t proc none /proc
> mount -t sysfs none /sys
> mount -t devtmpfs devtmpfs /dev
> echo
> echo "=====
> echo " minimal initramfs booted"
> echo " run: dmesg | grep NET-TCP"
> echo "=====
> echo
> exec /bin/sh
> EOF
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ chmod +x ~/android-kernel-lab/initramfs/init
```

4.打包 initramfs

```
cd ~/android-kernel-lab/initramfs
find . -print0 | cpio --null -ov --format=newc | gzip -9 > ../initramfs.cpio.gz
cd ..
ls -lh initramfs.cpio.gz
```

```
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ cd ~/android-kernel-lab/initramfs
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab/initramfs$ find . -print0 | cpio --null -ov --format=newc | gzip -9 > ../initramfs.cpio.gz
.
./sbin
./init
./bin
./bin/mount
./bin/busybox
./bin/ping
./bin/echo
./bin/cat
./bin/dmesg
./bin/sh
./dev
./proc
./usr
./usr/sbin
./usr/bin
./sys
./etc
4248 blocks
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab/initramfs$ cd ..
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$ ls -lh initramfs.cpio.gz
-rw-rw-r-- 1 ubuntu ubuntu 1.1M Apr 25 06:20 initramfs.cpio.gz
ubuntu@ubuntu-virtual-machine:~/android-kernel-lab$
```

2.2.6 使用 QEMU 启动

```
cd ~/android-kernel-lab
qemu-system-x86_64 \
```



```
-kernel arch/x86/boot/bzImage \
-initrd initramfs.cpio.gz \
-append "console=ttyS0 rdinit=/init loglevel=7" \
-nographic \
-m 1024
```

```
[ 0.000000] Linux version 5.15.202 (ubuntu@ubuntu-virtual-machine) (gcc (Ubu6
[ 0.000000] Command line: console=ttyS0 rdinit=/init loglevel=7
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x000000000009fbff] usable
[ 0.000000] BIOS-e820: [mem 0x000000000009fc00-0x000000000009ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000000f0000-0x00000000000fffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000003ffdffff] usable
[ 0.000000] BIOS-e820: [mem 0x00000000003ffe0000-0x00000000003fffffff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fffc0000-0x00000000ffffffff] reserved
[ 0.000000] NX (Execute Disable) protection: active
[ 0.000000] SMBIOS 2.8 present.
[ 0.000000] DMI: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.13.0-1ubuntu4
[ 0.000000] last_pfn = 0x3ffe0 max_arch_pfn = 0x400000000
[ 0.000000] x86/PAT: Configuration [0-7]: WB WC UC- UC WB WP UC- WT
[ 0.000000] found SMP MP-table at [mem 0x000f5ca0-0x000f5caf]
[ 0.000000] RAMDISK: [mem 0x3feda000-0x3ffdffff]
[ 0.000000] ACPI: Early table checksum verification disabled
[ 0.000000] ACPI: RSDP 0x000000000000f5ad 000014 (v00 BOCHS )
[ 0.000000] ACPI: RSDT 0x000000003ffe156f 000030 (v01 BOCHS  BXPGRSDT 000000)
[ 0.000000] ACPI: FACP 0x000000003ffe144b 000074 (v01 BOCHS  BXPGRFACP 000000)
[ 0.000000] ACPI: DSDT 0x000000003ffe0040 00140b (v01 BOCHS  BXPGRDSDT 000000)
[ 0.000000] ACPI: FACS 0x000000003ffe0000 000040
[ 0.000000] ACPI: APIC 0x000000003ffe14bf 000078 (v01 BOCHS  BXPGRAPIC 000000)
[ 0.000000] ACPI: HPET 0x000000003ffe1537 000038 (v01 BOCHS  BXPGRHPET 000000)
```

```
[ 4.566190] Freeing unused kernel image (initmem) memory: 1500K
[ 4.568584] Write protecting the kernel read-only data: 24576k
[ 4.573598] Freeing unused kernel image (text/rodata gap) memory: 2032K
[ 4.575628] Freeing unused kernel image (rodata/data gap) memory: 1192K
[ 4.577540] Run /init as init process
[ 4.727806] mount (45) used greatest stack depth: 14640 bytes left
[ 4.814349] input: ImExPS/2 Generic Explorer Mouse as /devices/platform/i8043
```

```
=====
minimal initramfs booted
run: dmesg | grep NET-TCP
=====
```

```
BusyBox v1.30.1 (Ubuntu 1:1.30.1-4ubuntu6.5) built-in shell (ash)
Enter 'help' for a list of built-in commands.
```

2.2.7 验证插桩日志

查看内核日志。在 QEMU 启动后的 shell 中执行，进行验证。

```
ifconfig lo 127.0.0.1 up
httpd -h / &
wget -O - http://127.0.0.1/
dmesg | grep NET-TCP
```

```
/ # ifconfig lo 127.0.0.1 up
[ 8.138112] ifconfig (48) used greatest stack depth: 14248 bytes left
```



```
/ # httpd -h / &
```

```
/ # wget -O - http://127.0.0.1/
Connecting to 127.0.0.1 (127.0.0.1:80)
[ 22.496845] TCP: [NET-TCP-SEND] pid=51 comm=wget size=72
[ 22.511266] TCP: [NET-TCP-RECV] pid=52 comm=httpd len=1024
[ 22.514323] TCP: [NET-TCP-RECV] pid=51 comm=wget len=4096
[ 22.527865] TCP: [NET-TCP-SEND] pid=52 comm=httpd size=231

/ # dmesg | grep NET-TCP
[ 22.496845] TCP: [NET-TCP-SEND] pid=51 comm=wget size=72
[ 22.511266] TCP: [NET-TCP-RECV] pid=52 comm=httpd len=1024
[ 22.514323] TCP: [NET-TCP-RECV] pid=51 comm=wget len=4096
[ 22.527865] TCP: [NET-TCP-SEND] pid=52 comm=httpd size=231
```

实验中在 `net/ipv4/tcp.c` 的 `tcp_sendmsg_locked()` 与 `tcp_recvmsg()` 中插入 `pr_info()` 日志，并重新编译内核。随后使用 QEMU 加载新内核与自制 `initramfs` 启动系统，在最小环境中配置回环接口、启动 BusyBox `httpd` 服务，并使用 `wget` 访问 `127.0.0.1`。通过 `dmesg | grep NET-TCP` 成功观察到 `[NET-TCP-SEND]` 与 `[NET-TCP-RECV]` 日志，日志中包含进程名、进程 ID 及发送/接收数据长度，说明网络访问行为已被内核插桩成功记录。

2.3 思考题总结

思考题通过搭建 Android 内核编译环境、下载并分析源码、对 TCP 网络收发关键函数进行插桩、重新编译并在 QEMU 中运行验证，完成了对 Android 底层关键功能的监测实验。通过实验可以看出，插桩是一种有效的系统行为观测方法，能够在网络访问、文件操作等核心路径上输出日志，为系统调试、安全分析和异常行为检测提供依据。同时我也认识到，Android 安全不仅体现在应用层权限控制，更依赖于内核层对关键资源访问过程的精细化监控。此次实验加深了我对 Android 内核结构、编译流程以及安全防护机制的理解，也提高了我对移动终端底层安全问题的分析能力。